

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

ALGORITMIA Y ESTRUCTURA DE DATOS

2da. práctica (tipo B)
(Primer Semestre 2026)

Duración: 1h 50 min.

- **No puede utilizar apuntes, solo hojas sueltas en blanco.**
- En cada función el alumno deberá incluir, a modo de comentario, la forma de solución que utiliza para resolver el problema. De no incluirse dicho comentario, el alumno perderá el derecho a reclamo en esa pregunta.
- No puede emplear plantillas o funciones no vistas en los cursos de programación de la especialidad.
- Los programas deben ser desarrollados en el lenguaje C++. Si la implementación es diferente a la estrategia indicada o no la incluye, la pregunta no será corregida.
- Un programa que no muestre resultados coherentes y/o útiles será corregido sobre el 50% del puntaje asignado a dicha pregunta.
- Debe utilizar comentarios para explicar la lógica seguida en el programa elaborado. El orden será parte de la evaluación.
- **Solo está permitido acceder a la plataforma de PAIDEIA, cualquier tipo de navegación, búsqueda o uso de herramientas de comunicación se considera plagio por tal motivo se anulará la evaluación y se procederá con las medidas disciplinarias dispuestas por la FCI.**
- Para esta evaluación solo se permite el uso de las librerías `iostream`, `iomanip`, `limits`, `cstring`, `ctime`, `cstdlib`, `cmath` o `fstream`. Recuerde que no puede emplear variables globales.
- Su trabajo deberá ser subido a PAIDEIA.
- **Es obligatorio usar como IDE CLion. No está permitido el uso de variables globales.**
- Los archivos deben llevar como nombre su código de la siguiente forma `codigo_LAB2_P#` (donde # representa el número de la pregunta a resolver)

Pregunta 1 (10 puntos)

Barajado de una baraja usando listas enlazadas simples

Se desea modelar y barajar una baraja de naipes utilizando una **lista enlazada simple**. Cada nodo representa una carta y contiene la información del número y el palo.

Estructuras

Adjunto a este enunciado, se cuenta con un archivo llamado **“Estructuras.h”**, su programa debe soportar el uso de estas.

Funciones de la librería

Se les proporciona una librería con funciones básicas de listas enlazadas (inicialización, inserción al final, verificación de lista vacía, etc.). Puede modificar las funciones de la librería proporcionada para soportar el diseño del problema.

Parte 1: Preparación del problema (3 puntos)

Implemente la función: `void crear_baraja(Baraja &baraja);`

Esta función debe: Inicializar la baraja. Insertar las **52 cartas** (4 palos × 13 números). Mantener correctamente los punteros y la longitud.

Parte 2: Resolución del problema (7 puntos)

Función 1: extracción por posición

Implemente la función: `NodoBaraja* extraer_en_posicion(Baraja &baraja, int posicion);`

Esta función debe: Extraer el nodo en la posición indicada. Retornar el nodo extraído. Actualizar los punteros correctamente. Desconectar el nodo de la lista original. En todo momento debe trabajar sobre la misma baraja.

Función 2: barajado

Implemente la función: **void barajar(Baraja &baraja);**

Descripción del algoritmo de barajado

El barajado se realiza **sin crear una nueva lista**, reorganizando los nodos dentro de la misma baraja.

El algoritmo sigue la siguiente idea:

1. Se considera que la baraja tiene dos zonas:
 - o una **zona pendiente** por barajar
 - o una **zona ya barajada** (al final de la lista)
2. Mientras existan cartas **pendientes**:
 - o Se genera una posición **aleatoria** válida dentro de la zona **pendiente**.
 - o Se **extrae** la carta en esa posición.
 - o Se **inserta al final** de la baraja.
3. La zona **pendiente** se **reduce** progresivamente hasta que toda la baraja queda barajada.

Función 3: destrucción

Implemente la función: **void destruir(Baraja &baraja);**

Esta función debe eliminar todos los nodos y conexiones de la baraja.

Restricciones

- No se permite crear nuevos nodos durante el barajado.
- No se permite usar arreglos ni estructuras auxiliares.
- Solo se deben manipular punteros.
- Se debe utilizar generación de números aleatorios (puede usar **srand(time(NULLptr))** y **rand()** dentro de **<cstdlib>** y **<ctime>**).
- La salida no será única debido al uso de aleatoriedad.
- Debe **obligatoriamente destruir** la baraja al final del programa.
- La evaluación se centrará en la **correcta manipulación de la lista enlazada y el algoritmo aplicado**, no en el orden final de las cartas.

Ejemplo de Salida del Programa

```
=====
BARAJA ORIGINAL
=====
1C 2C 3C 4C 5C 6C 7C 8C 9C 10C 11C 12C 13C 1D 2D 3D 4D 5D 6D 7D 8D 9D 10D 11D 12D 13D 1T 2T 3T 4T 5T 6T 7T 8T 9T 10T 11T 12T 13T 1E 2E 3E 4E 5E 6E 7E 8E 9E 10E 11E 12E 13E

=====
BARAJADO
=====
12C 1E 9D 13T 10C 1C 2T 5D 7E 6D 1D 5T 7T 11E 9T 8T 11C 11T 8D 11D 5C 2C 6T 6C 8C 13D 5E 3T 6E 8E 4D 12E 3D 10D 3E 13C 12D 2E 4T 12T 4E 2D 9C 4C 3C 7C 10T 9E 1T 10E 13E 7D

=====
LIBERAR MEMORIA
=====
BARAJA VACIA
```

Pregunta 2 (10 puntos)

En el reconocido aviario Fénix Aviario se ha iniciado una importante investigación genética destinada a encontrar al **canario ancestral**, aquel ejemplar cuya línea hereditaria permanece más cercana al origen de toda la población actual. Durante años, los especialistas del aviario han registrado cuidadosamente los cruces y aportes genéticos entre sus canarios. Como resultado, cuentan con un registro donde se indica la cantidad de material genético que un canario ha transmitido a otro.

Según los expertos, el verdadero canario ancestral debe cumplir una característica muy especial: **ha aportado genes a todos los demás ejemplares del aviario y no ha recibido aportes genéticos de ninguno de ellos (excepto el mismo)**, conservando así una línea pura y original.

Por tal motivo se necesita que desarrolle un programa que, a partir del registro de aportes genéticos, determine cuál de los canarios es el canario ancestral. Si existe un único candidato, deberá mostrarse su número asignado; en caso contrario, si ningún ejemplar cumple las condiciones, se deberá mostrar -1.

Para resolver este problema, la solución debe ser implementada utilizando recursión, puede usar más de una función. El desarrollo debe tener una complejidad máxima $O(n)$. Para que su solución sea válida no debe usar estructuras auxiliares o iterativas.

A continuación, se muestra el registro genético:

Registro de Aportes

	0	1	2	3	4	5	6	7	8	9
0	100	0	50	40	30	20	30	0	80	0
1	50	100	0	40	30	20	20	0	10	25
2	80	30	100	40	30	0	30	20	10	60
3	50	0	0	100	30	0	50	30	30	90
4	50	10	10	10	100	0	10	50	10	50
5	20	0	0	0	0	100	90	20	40	20
6	0	0	0	0	0	0	100	0	0	0
7	0	0	0	0	0	0	50	100	50	20
8	20	0	0	40	0	0	90	0	100	10
9	0	10	0	0	0	0	10	0	60	100

Lease la posición Aporte[4,2], el canario 2 aporta el 10% de sus genes al canario 4.

Lease la posición Aporte[5,6], el canario 6 aporta el 90% de sus genes al canario 5.

Para la matriz del ejemplo la solución es el canario 6.

Al finalizar el laboratorio, comprima la carpeta de su proyecto empleando el programa Zip que viene por defecto en el Windows, **no se aceptarán los trabajos compactados con otros programas como RAR, WinRAR, 7zip o similares**. Luego súbalo a la tarea programa en Paideia para este laboratorio.

Profesores del curso:

Ana Roncal
Fernando Huamán
David Allasi
Jose Luis Corcuera
Rony Cueva
Erasmó Gomez

San Miguel, 2 de mayo del 2026